

Rapport de TD n°4: Initiation à l'algorithmique

Derrez Lorenzo

September 30, 2026

Exercice 1

Question 1:

On a l'équation,

$$\#(n) = 1 + 2\#(\frac{n}{2}).$$

Donc,

- $a = 2$ appels récursifs.
- $b = 2$ la division de la taille de l'entrée.
- $k = 0$ l'exposant de $\#division + \#recomposition$.

Par le cours, on obtient (cas où $a > b^k$),

$$\theta(n^{\log_b(a)})$$

Finalement la solution est ; $\theta(n)$.

Optimalité : La solution donnée est en θ donc elle est optimale.

Question 2:

- Une définition naïve récursive est, pour un tableau T de taille n ;
$$LongMaxPlateau(n) = \max(LongMaxPlateau(n_{gauche}), LongMaxPlateau(n_{droit}))$$
- C'est une définition erronée car :
elle ne s'applique pas correctement pour une entrée contenant la longueur maximale du plateau au milieu de T , et non au début ou à la fin de T .
- Par exemple : $T = [2, 1, 2, 2, 2, 1, 2], n = 8$.
- La définition récursive naïve va ici renvoyer le max entre les tableaux gauche et droit. Or ces deux appels vont valoir 2 et dans ce cas, l'output renvoyé sera différent de 4 qui est l'output attendu pour cet exemple.

Question 3:

Entrée : T un tableau d'entiers, et deux indices $i, j \leq 2$ qui délimitent la zone $T[i..j[$.

Sortie : un triplet (m_g, m, m_d) où,

- m_g est la longueur du plateau collé au début de la zone (il commence en i et finit en $i + m_g - 1$).
- m est la longueur maximale d'un plateau dans la zone
- m_d est la longueur du plateau collé à la fin de la zone (il commence en $j - m_d$ et finit en $j - 1$).

Le milieu est $c = (i + j) // 2$. On a deux appels,

- $S = (m_g, m, m_d)$ pour la zone gauche $[i, c[$ de taille $c - i$.

- $U = (m_g, m, m_d)$ pour la zone droite $[c, j[$ de taille $j - c$.

Cas par défaut (aucun plateau ne traverse le milieu),

- $m_g = m_{g_1}$
- $m = \max(m_1, m_2)$
- $m_d = m_{d_2}$

Si $T[c-1] = T[c]$ (un plateau traverse le milieu), on corrige :

- $m = \max(m_1, m_2, m_{d_1} + m_{g_2})$
- si $m_{g_1} = c - i$ (la gauche est entièrement un plateau), alors $m_g = m_{g_1} + m_{g_2}$
- si $m_{d_2} = j - c$ (la droite est entièrement un plateau), alors $m_d = m_{d_1} + m_{d_2}$

Question 4:

Listing 1: `lmpRec`

```

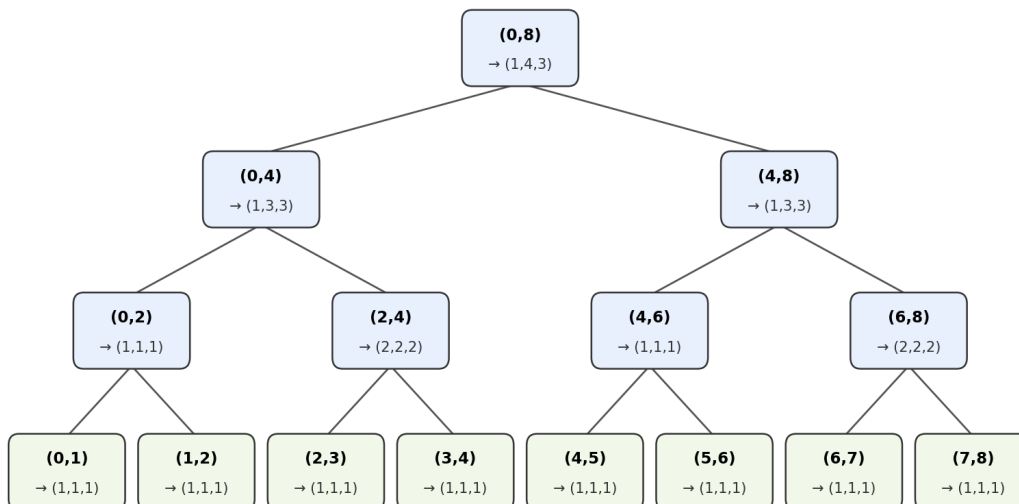
1 def lmpRec( T : Array[integer], i : integer, j : integer)-> (integer, integer, integer):
2   if j-i < 2:
3     return (j-i, j-i, j-i) #zone vide ou 1 element
4   c = (i+j)//2
5   S = lmpRec(T, i, c) # (mg1, m1, md1)
6   U = lmpRec(T, c, j) # (mg2, m2, md2)
7   mg, m, md = S[0], max(S[1], U[1]), U[2]
8   if T[c-1] == T[c]:
9     m = max(m, S[2] + U[0])
10    if S[0] == c-i:
11      mg = S[0] + U[0]
12    if U[2] == j-c:
13      md = U[2] + S[2]
14   return (mg, m, md)

```

Question 5:

Pour réaliser l'arbre d'appels en entier, on peut l'écrire de la manière suivante :

image_exo1_algo_TD4



Question 6:

Complexité en espace :

- Déterminer la capacité maximale utilisée à un instant de l'exécution.
- Durant chaque appel, on stock :
 $S, U, i, j, mg, md \rightarrow \theta(1)$
- S et U ne sont pas simultanés :
on regarde le max de la mémoire, pas la somme.
- L'espace se définit pour n la taille de l'entrée :

$$E(n) = (\text{espace local}) + \max(E(\frac{n}{2}), E(\frac{n}{2}))$$

Entre autres,

$$E(n) = 1 + E(\frac{n}{2}) \times 1$$

où $a = 1, b = 2, k = 0$.

Par le cours, on obtient,

$$\theta(\log(n)) \text{ en espace.}$$

Par ailleurs, il y a $1 + \log_2(n)$ appels.

Exercice 2

Question 1:

Exercice 2;

Q1/

- Jeu d'exemple pour le problème Fusion;

$\left\{ \begin{array}{l} E: T = [3, 1, 2] \\ S: U = [1, 2, 3] \end{array} \right. , a = 0, b = 1, c = 3$

$\left\{ \begin{array}{l} E: T = [6, 8, 1, 3, 4] \\ S: U = [1, 3, 4, 6, 8] \end{array} \right. , a = 0, b = 2, c = 5$

$\left\{ \begin{array}{l} E: T = [7, 8, 9, 10, 5, 6] \\ S: U = [5, 6, 7, 8, 9, 10] \end{array} \right. , a = 0, b = 4, c = 6$

$\left\{ \begin{array}{l} E: T = [1, 2, 3, 4] \\ S: U = T = [1, 2, 3, 4] \end{array} \right. , a = 4, b = 4, c = 4$

$\left\{ \begin{array}{l} E: T = [3] \\ S: U = T = [3] \end{array} \right. , a = 1, b = 1, c = 1$

Figure 1: Jeu d'exemples pour le problème de FUSION.

Question 2

Algorithm 1 Solution Algorithmique de *FUSION*

```
1: Entrée : un tableau T d'entiers, a, b et c des entiers vérifiant :  $0 \leq a \leq b \leq c \leq \text{size}(T)$ 
2: Sortie : un tableau U d'entiers tel que
    •  $U[0:a] = T[0:a]$ 
    •  $U[c : |U|] = T[c : |T|]$ 
    •  $U[a : c]$  et  $T[a : c]$  ont memes valeurs (avec memes nombre d'occurrence)
    • Si  $T[a:b]$  et  $T[b:c]$  sont triés alors  $U[a:c]$  est trié.
3: function FUSION((T : Array[integer], a:int, b:int, c:int) -> Array[integer]):
4:   int n  $\leftarrow$  size(T)
5:   U  $\leftarrow$  newArray(n, 0)
6:   int k  $\leftarrow$  0
7:   Tant que k < a :
8:     U[k]  $\leftarrow$  T[k]
9:     k  $\leftarrow$  k + 1
10:  int i  $\leftarrow$  a
11:  int j  $\leftarrow$  b
12:  Tant que i < b et j < c:
13:    Si T[i]  $\leq$  T[j]:
14:      U[k]  $\leftarrow$  T[i]
15:      i  $\leftarrow$  i + 1
16:    Sinon
17:      U[k]  $\leftarrow$  T[j]
18:      j  $\leftarrow$  j + 1
19:    k  $\leftarrow$  k + 1
20:  Tant que i < b:
21:    U[k]  $\leftarrow$  T[i]
22:    k  $\leftarrow$  k + 1
23:    i  $\leftarrow$  i + 1
24:  Tant que j < c:
25:    U[k]  $\leftarrow$  T[j]
26:    j  $\leftarrow$  j + 1
27:    k  $\leftarrow$  k + 1
28:  Tant que k < n :
29:    U[k]  $\leftarrow$  T[k]
30:    k  $\leftarrow$  k + 1
31:  return U
32: end function
```

La complexité est en $\theta(n)$ en temps et en espace, où n est la taille de T .

Question 3:

- Actuellement, à chaque appel de la fonction *FUSION*, on crée un nouveau tableau U.
- On pourrait créer une fonction *TRI_FUSION* dans laquelle on crée U une seule fois et on utilise *FUSION* comme une fonction auxiliaire avec U en paramètre.
- Cette méthode réduit le nombre d'appels à la création de U.
- Toutefois, la complexité en espace reste $\theta(n)$ car il s'agit de la mémoire maximale utilisée lors de l'exécution et non le nombre total de créations, et la création de U est linéaire en n .

Exercice 3

Question 1:

* Exercice 3;

Q1/ Jeu d'exemples pour le problème «Decomposition - Pivot»;

$\left\{ \begin{array}{l} E: T=[1,6,5,4,3]; a=0; b=5 \\ S: i=0; U=[1,3,4,5,6] \end{array} \right.$

$\left\{ \begin{array}{l} E: T=[1,6,5,4,3]; a=0; b=5 \\ S: i=0; U=[1,6,5,4,3] \end{array} \right.$

$\left\{ \begin{array}{l} E: T=[1,6,5,4,3]; a=0; b=5 \\ S: i=2; U=[1,3,4,5,6] \end{array} \right.$

$\left\{ \begin{array}{l} E: T=[1,2,1,3]; a=0; b=4 \\ S: i=3; U=[1,2,1,3] \end{array} \right.$

$\left\{ \begin{array}{l} E: T=[1,2,1,3]; a=0; b=4 \\ S: i=2; U=[1,1,2,3] \end{array} \right.$

$\left\{ \begin{array}{l} E: T=[3,4,7,2,8,1]; a=1; b=5 \\ S: i=3; U=[3,2,4,7,8,1] \end{array} \right.$

Figure 2: Jeu d'exemples pour le problème de DECOMPOSITION_PIVOT.

Question 2:

Listing 2: DECOMPOSITION_PIVOT

```
1 def decomposition_pivot(T: Array[integer], a: integer, b: integer) -> integer :  
2     indPivot = indice_pivot(T, a, b)  
3     p = T[indPivot]  
4     swap(T, a, indPivot)  
5     i = a  
6     j = b  
7     while i+1 < j :  
8         if T[i] > T[i+1]:  
9             swap(T, i, i+1)  
10            i = i + 1  
11        else:  
12            swap(i+1, j-1)  
13            j = j - 1  
14    return i
```

Question 3:

Pour la complexité temporelle :

- La boucle fait $b-a-1$ tours.
- Chaque tours coûte $\theta(1)$ (une comparaison, un swap coûtant $\theta(1)$ et une affectation).

Donc, pour $b - a = n$ la taille de l'entrée, la complexité temporelle est :

$$\theta(n)$$

Pour la complexité en espace :

– Seules quelques variables entières sont ajoutées et T est modifié en place.

Donc la complexité en espace est : $\theta(1)$